WORK PERFORMANCE REPORT

The Value Delivered — Single-Handed Full-Stack Delivery

TaskNest | TMS | PMS | In-House AI

Employee	Qasim (ninja00707)
Role	Full-Stack Developer & System Designer
Report Period	May 2026 – August 2026 (ongoing)
Status	TaskNest LIVE & deployed | TMS ~90% | PMS ~90%

This report documents the development work delivered **single-handedly** over the reporting period. It focuses on the **value** created: from a blank repository, the flagship **TaskNest** enterprise platform was architected, built, and taken to **LIVE production** by myself alone, while the **TMS** (~90%) and **PMS** (~90%) systems were progressed in parallel, and an **in-house AI** initiative is now in planning for a ~6-month maturation cycle.

Purpose of this document: To show the scale and value of what one person delivered — the full scope of engineering, the production result, and the forward-looking AI roadmap — so the effort and worth of this work are fully visible and understood.

Note: Figures for TaskNest are derived from the actual source repository (git history, file and line counts, module inventory). Figures marked as estimates (est.) for TMS/PMS should be reconciled with the relevant project leads at review time.

1. Executive Summary

Over the reporting period I acted as the **sole developer, architect, and owner** of a substantial software portfolio — every line was written, every decision taken, and every deployment executed by one person. The flagship product, **TaskNest** — an enterprise multi-company, multi-department ticketing and task management platform — was taken from a blank repository to a **production-LIVE, deployed application** in approximately three months. This involved a full-stack build across a Flutter frontend and a Node.js/PostgreSQL backend, with real-time communication, an admin panel, multi-department workflows, disputes, projects, and role-based dashboards. Two further systems, **TMS** (~90%) and **PMS** (~90%), were progressed in parallel, and an **in-house AI** initiative is now in planning for a ~6-month maturation cycle.

Headline delivery metrics (TaskNest, measured from repository):

Metric	Value	Metric	Value
Total commits	127 (sole contributor)	Active development	~3 months
utter source files	201 Dart files	Flutter lines of code	~38,000
kend source files	44 JS files	Backend lines of code	~7,200
atabase schema	30 SQL files	Multi-platform targets	Android/iOS/Web + D
Modules	11 feature modules	Core feature areas	12+ modules
eployment status	LIVE / in production	Avg activity	~10 commits / we

2. The Value of a Single Hand

Everything listed in this report was designed, coded, tested, and deployed **by me alone** — no development team, no external contractor. The value of this work lies not just in what was built, but in what one person carried end-to-end:

- **Complete self-sufficiency:** One resource delivered the equivalent of a full development team — frontend, backend, database, and infrastructure.
- **Full ownership, zero hand-offs:** No context loss between requirements, design, and code because the same person held every stage.
- **Cost value:** The entire product portfolio was produced without external development or consulting spend.
- **Production delivery:** TaskNest is **LIVE and deployed** — a real, working system of record, delivered personally.
- **Real-time complexity mastered:** Socket.IO with JWT auth, Redis scaling, and an outbox pattern for reliable notifications.
- **Business-logic depth:** Multi-company isolation, cascading sub-tickets, approvals, reopen rules, and dispute handling — all solo.
- **Parallel capacity:** TMS and PMS advanced to ~90% each while TaskNest was carried to live — high throughput from one person.

- **Forward-looking AI:** Built an LLM-driven agent (Alpaca + Ollama) and is driving the in-house AI effort into the stack.

3. Flagship Project — TaskNest (LIVE)

3.1 Project Overview

TaskNest is an enterprise **ticketing and task-management platform** for multi-company, multi-department organisations. It replaces fragmented tracking with a single source of truth, role-aware dashboards, and real-time collaboration — covering UM Enterprises and Matrix Pharma (company-level isolation) with a flexible department hierarchy.

3.2 Technology & Architecture

Frontend (Flutter/Dart)	Backend (Node.js / PostgreSQL)
Flutter SDK ^3.11.5, flutter_bloc (BLoC state)	Express ~4.16 REST API
go_router declarative routing + auth guards	PostgreSQL (pg ^8.20) relational schema
dio HTTP client, get_it + injectable DI	Socket.IO ^4.8 real-time (JWT auth)
socket_io_client v3.0.2 — real-time UI	Redis (ioredis) + BullMQ queue / caching
shared_preferences local storage	JWT + bcrypt auth, rate limiting
Multi-target: Android/iOS/Web/Windows/Linux/macOS	xlsx import/export, cluster mode, PM2

The codebase follows **clean architecture**: a layered frontend (data > domain > presentation with compile-time DI via injectable), and a controller > service > repository backend, with a **database outbox pattern** for reliable notification delivery.

3.3 Delivered Features (Core Value)

Authentication & Users

- Login by employee code / admin email with JWT sessions
- Signup with admin approval workflow
- Forgot password with 6-digit reset code (email, 15-min expiry)
- First-login forced password reset
- Role model: CEO / Manager / Employee
- Multi-company isolation (UM Enterprises, Matrix Pharma)

Ticket Management (core engine)

- Create tickets (title, description, priority, due date)
- Multi-department (multi_task) tickets with sub-tickets per department
- Cross-department transfers and independent progress tracking
- Status workflow open > in_progress > closed
- Self-assignment & manager assignment
- Sub-ticket cascading: parent auto-closes when all children close
- Reopen workflow with rules (creator-only, 1 max, 48h window)
- Sub-department approvals (pending_approval > approved)
- Full audit log (created/assigned/transferred/closed/reopened/disputed)

Disputes

- Raise disputes: wrong_resolution, sla_breach, wrong_assignment, quality_issue, other
- Review workflow: under_review > resolved / rejected / escalated
- Withdraw disputes, dispute comments, activity tracking
- Dispute can force-close the entire ticket tree

Projects

- Project creation, listing, and association with tickets (optional projectId)

Dashboard & Analytics

- Role-based dashboards: CEO (org-wide), Manager (department), Employee (personal)
- Ticket stats grids, priority breakdowns, resolution metrics
- Recent activities timeline with status dots
- Redis-cached stats for performance

Notifications & Real-time

- Socket.IO live notifications, unread badge counter
- Dedup — same message for a ticket never repeated
- Database outbox > processor > socket emit (reliable delivery)
- Department-scoped broadcasts (dept_{id} rooms)

Navigation & UI

- Responsive: sidebar (desktop/web), bottom nav (mobile)
- GoRouter ShellRoute + auth guards, clean URLs
- Kanban board, list, and grid views with search/filter/pagination
- 18+ reusable theme/style widgets for UI consistency

Admin Panel

- Full user CRUD: create, update, delete, approve, deactivate
- Hierarchical department management with parent-child
- Ticket management with admin override, user activity tracking
- Reporting structure (reports_to), shared-department option, see_all_companies flag

3.4 Effort & Scale Evidence

The numerical evidence below is drawn directly from the TaskNest repository and git history.

Scope metric	Measured value
Total commits	127
Commits contributed	127 / 127 (100% — sole author)
Active development window	16 May – 18 Aug 2026 (~3 months)
Commits by month	May 55 · Jun 42 · Jul 24 · Aug 6
Flutter codebase	201 Dart files / ~38,000 lines

Backend codebase	44 JS files / ~7,200 lines
Database	30 SQL files (schema + migrations v3–v11)
Feature modules	11 (admin, tickets, dashboard, disputes, projects, login, etc.)

Work was delivered in rapid, incremental cycles — from login and dashboard through to sub-ticket workflows, disputes, and multiple **LIVE deployments** — reflecting both speed and iterative quality control.

4. Parallel Systems — TMS & PMS (≈90% complete)

In parallel with TaskNest, two further systems were designed, built, and brought to approximately 90% completion: **TMS** (Transport/Ticket Management System) and **PMS** (Project/Process Management System). The following rows capture the effort involved; complete the highlighted quantitative fields where specific numbers are available.

Aspect	TMS	PMS
Status	~90% complete	~90% complete
Primary purpose	Manage the organisation's transport / ticket operations	Manage projects / processes across the business
Key modules built	Core data & workflow modules implemented; ~10% remaining (final integration/polish)	Core data & workflow modules implemented; ~10% remaining (final integration/polish)
Development effort	Significant — designed, built, and unit-tested in parallel with TaskNest (est.)	Significant — designed, built, and unit-tested in parallel with TaskNest (est.)
Completion plan	Remaining 10% scheduled; integration with live infrastructure	Remaining 10% scheduled; integration with live infrastructure

The concurrent delivery of three systems — one of which is fully LIVE — demonstrates high workload capacity, prioritisation, and sustained output over the period.

5. In-House AI Initiative

A significant forward-looking effort is the **in-house AI** capability, planned to be integrated into the platform over a **~6-month maturation cycle**, followed by several further months of tuning and validation for reliable results. This is a strategic investment that will differentiate the product.

5.1 Proof-of-Concept Already Delivered

An AI-driven trading agent has already been designed and implemented in Python: it consumes market data via the Alpaca API, runs analysis through a local **Ollama LLM** (“Cool Analyst”), and generates structured buy/sell/hold decisions with confidence scores and reasoning — proving the capacity to design, build, and run LLM-powered automation end-to-end.

5.2 Planned Roadmap (est.)

Phase	Focus	Timeframe
Setup & scoping	Define AI features for the platform; data pipeline design	Month 1
Model & data foundation	Data collection, feature engineering, model selection	Month 2–3

Maturation	Training, evaluation, and iterative improvement of the in-house model	Month 4–6
Integration	Wire the matured AI into TaskNest/TMS/PMS flows	Following months
Validation & release	Tuning, accuracy targets, controlled rollout for reliable results	Following months

This represents real, ongoing engineering investment and positions the organisation ahead on AI-enablement rather than merely following it.

6. Net Value Delivered — At a Glance

The table below distils, at a glance, the value one person delivered across the period.

#	Area	Value Delivered (single-handed)
1	Production result	TaskNest shipped **LIVE and deployed** in ~3 months — a real, usable system of record, not a prototype.
2	Scale of output	127 commits; ~45,000+ lines; 201+44 source files — produced by one person.
3	Full-stack breadth	Frontend, backend, DB design, real-time infra, security, admin tooling, DevOps — all covered solo.
4	Complexity handled	Multi-company isolation, cascading sub-tickets, approvals, reopen rules, disputes, real-time outbox pattern.
5	Parallel capacity	TMS and PMS advanced to ~90% each while carrying TaskNest to live.
6	Strategic direction	Driving the in-house AI roadmap (~6-month maturation); an LLM PoC already built and running.
7	Zero external cost	The whole portfolio was produced without external development or consulting spend.

7. Conclusion

The value summarised above is substantial, shipped, and **in production today** — and it was delivered by a single person. Command of the full stack, delivery of a complex enterprise product solo, concurrent progression of two further systems to ~90%, and an active in-house AI initiative together represent a return far beyond a single-ops salary line. This document lays out that value clearly so the scale of the contribution is fully recognised.

Prepared by: Qasim (ninja00707) **Date:** 2 September 2026

This report was auto-generated from live repository metrics and developer-supplied project status. Figures for TMS/PMS with (est.) should be finalised with project data where available.